

Báo cáo Quá trình Tái cấu trúc và Triển khai Data Pipeline

Hệ thống Phân tích và Dự báo Năng lượng Mặt trời

Ngo Tan Dat

Ngày 21 Tháng 06 Năm 2026

Mục lục

1	Hiện trạng và Vấn đề Ban đầu	3
2	Quá trình Tái cấu trúc (Refactoring)	3
2.1	Tham số hóa toàn hệ thống bằng YAML	3
2.2	Điều phối trung tâm bằng Argparse	4
2.3	Tối ưu hiệu năng truyền tải Cloud (StringIO & COPY)	4
2.4	Kiểm soát an toàn Kiểu dữ liệu kép	5
3	Đối chiếu và Nghiệm thu Hệ thống	5

1 Hiện trạng và Vấn đề Ban đầu

Trong giai đoạn đầu, toàn bộ quy trình ETL (Extract - Transform - Load) được thiết kế dưới dạng nguyên khối (Monolithic). Cấu trúc này bộc lộ 3 vấn đề kỹ thuật lớn khi triển khai thực tế:

- **Khó cấu hình (Hardcode):** Các thông số kết nối, tài khoản, và logic nghiệp vụ bị gắn chặt vào mã nguồn, gây khó khăn lớn khi di chuyển từ môi trường phát triển (Local) lên hệ thống thực tế (Cloud).
- **Nghẽn cổ chai mạng (Network Bottleneck):** Phương thức nạp dữ liệu tiêu chuẩn gặp tình trạng quá tải (Timeout) khi phải đẩy hàng triệu bản ghi lên máy chủ Cloud trong một phiên kết nối.
- **Bất đồng bộ kiểu dữ liệu (Type Casting):** Khung dữ liệu thô gặp lỗi xung đột định dạng (ví dụ: chuỗi ký tự thập phân không thể tự động nhận diện thành số nguyên) khi luân chuyển qua lại giữa các lớp Datawarehouse.

2 Quá trình Tái cấu trúc (Refactoring)

Để tối ưu hóa luồng dữ liệu, hệ thống được đập đi xây lại theo mô hình chuẩn Modular, phân tách thành 4 module độc lập: 01_extract, 02_transform, 03_load, và 04_build_data_marts.

2.1 Tham số hóa toàn hệ thống bằng YAML

Loại bỏ hoàn toàn kỹ thuật hardcode. Mọi tham số cấu hình được chiết xuất và quản lý độc lập tại thư mục config/.

```
1 import yaml
2 from pathlib import Path
3
4 CONFIG_PATH = Path("config/04_machine_learning/01
   _bi_mart_params.yaml")
5 with CONFIG_PATH.open() as f:
6     config_data = yaml.safe_load(f)
7
8 SOURCE_SCHEMA = config_data["database"]["source_schema"]
9 TARGET_SCHEMA = config_data["database"]["target_schema"]
10 FIT_RATE = config_data["financial"]["fit_rate_vnd_per_kwh"]
```

Listing 1: Đọc tham số động từ cấu hình YAML

2.2 Điều phối trung tâm bằng Argparse

Xây dựng một tệp điều khiển duy nhất (`main.py`), cung cấp giao diện dòng lệnh linh hoạt để kỹ sư có thể kích hoạt từng khâu xử lý riêng biệt.

```
1 import argparse
2
3 parser = argparse.ArgumentParser(description="Solar Pipeline
4                                     Orchestrator")
5 parser.add_argument("--stage", type=str, required=True,
6                     choices=["extract", "transform", "load", "
7                             marts"])
8 args = parser.parse_args()
9
10 if args.stage == "load":
11     run_load_datawarehouse(engine)
```

Listing 2: Điều phối luồng hệ thống bằng Argument Parser

2.3 Tối ưu hiệu năng truyền tải Cloud (StringIO & COPY)

Để giải quyết dứt điểm lỗi đứt kết nối mạng do quá tải INSERT, hệ thống áp dụng kỹ thuật kết hợp luồng dữ liệu nhị phân trên RAM (`StringIO`) và câu lệnh `COPY` siêu tốc của PostgreSQL. Tốc độ nạp 2.7 triệu dòng được rút ngắn chỉ còn vài chục giây.

```
1 import io
2
3 def bulk_upload_fast(df, table_name, schema, engine):
4     buffer = io.StringIO()
5     df.to_csv(buffer, index=False, header=False)
6     buffer.seek(0)
7
8     raw_conn = engine.raw_connection()
9     try:
10         with raw_conn.cursor() as cursor:
11             cursor.copy_expert(f"COPY {schema}.{table_name}
12                                FROM STDIN WITH (FORMAT CSV)", buffer)
13         raw_conn.commit()
14     finally:
15         raw_conn.close()
```

Listing 3: Cơ chế nạp dữ liệu siêu tốc lên Cloud

2.4 Kiểm soát an toàn Kiểu dữ liệu kép

Giải quyết dứt điểm các xung đột về định dạng dữ liệu (ví dụ: cột `hour` chứa chuỗi "1.0" cần chuyển sang `INTEGER`) bằng kỹ thuật ép kiểu bảo mật nhiều lớp ngay trên truy vấn CSDL:

```
1 SELECT DISTINCT
2     CAST(CAST(t.hour AS FLOAT) AS INT) as hour ,
3     CAST(CAST(t.minute AS FLOAT) AS INT) as minute
4 FROM staging.dim_time t;
```

Listing 4: Ép kiểu 2 lớp trong Pipeline

3 Đối chiếu và Nghiệm thu Hệ thống

Giai đoạn kiểm thử cuối cùng thực hiện việc đối chiếu (Cross-validation) toàn bộ kiến trúc bảng giữa hai môi trường: máy trạm phát triển (**Local**) và máy chủ dữ liệu thực tế (**Cloud Supabase**).

Hệ thống ghi nhận sự đồng bộ hoàn hảo:

Lược đồ & Tên Bảng	Dữ liệu Local	Dữ liệu Cloud	Kết quả
staging.fact_solar_energy_gen	2,731,946	2,731,946	Khớp 100%
datawarehouse.fact_solar_energy_gen	2,731,946	2,731,946	Khớp 100%
datawarehouse.dim_solar_site	42	42	Khớp 100%
datawarehouse.dim_geography	42	42	Khớp 100%
datawarehouse.dim_date	2,312	2,312	Khớp 100%
datawarehouse.dim_time	96	96	Khớp 100%
datawarehouse.dim_weather_type	22	22	Khớp 100%
datawarehouse.fact_weather	850,752	850,752	Khớp 100%
bi_mart.fact_solar_performance_hourly	682,542	682,542	Khớp 100%

Bảng 1: Bảng Validation đối chiếu số lượng bản ghi sau tái cấu trúc

Tổng kết: Quy trình Data Pipeline đã đạt chuẩn Modular vững chắc, khắc phục hoàn toàn rào cản truyền tải trên Cloud, và sẵn sàng đóng vai trò xương sống cho việc phân tích dữ liệu chuyên sâu của tổ chức.